CS432 Assignment 3 Report

Module A: Storage Engine

Module B: Concurrent Workload and ACID Transactions

GitHub Repository Link:
`https://github.com/1357koushik/Sharkey_Database_Project/tree/main/Assignment_3`

Video Link:
`https://drive.google.com/file/d/1f2_W_fQBMqoptp1lSOA9EcyDUejfk0GO/view?usp=sharing`

1. Report Overview

This report is organised to match the assignment checklist:

1. Correctness of operations
2. Failure handling and recovery
3. Multi-user conflict handling
4. Experiments performed
5. Observations and limitations

The notebook also includes short executable demonstrations for the key Module A mechanisms so the report is not only descriptive but also reproducible.

2. Environment Setup

Run the next cell first. It adds `Module_A/` to the Python path and imports the classes used in the demonstrations.

```
import json
import os
import sys
import tempfile
import threading
import time
from pathlib import Path

REPO_ROOT = Path.cwd()
MODULE_A = REPO_ROOT / "Module_A"

if str(MODULE_A) not in sys.path:
    sys.path.insert(0, str(MODULE_A))
```

```python
from bplustree import BPlusTree
from db_manager import DatabaseManager
from transaction import _get_lock


def temp_wal_path():
    fd, path = tempfile.mkstemp(suffix=".wal")
    os.close(fd)
    return path


def cleanup_path(path):
    if path and os.path.exists(path):
        os.remove(path)


print("Repository root:", REPO_ROOT)
print("Module_A path:", MODULE_A)
print("Imports loaded successfully.")
```

```
Repository root: d:\2nd SQL\project\Sharkey_A3
Module_A path: d:\2nd SQL\project\Sharkey_A3\Module_A
Imports loaded successfully.
```

3. Module A Storage Engine

3.1 Why a B+ Tree?

The project uses a B+ Tree of order 4 as the core storage structure. This gives:

- `O(log n)` search, insert, and delete
- efficient sequential scans through linked leaf nodes
- efficient range queries with `O(log n + k)` complexity

3.2 Key design choices

- Internal nodes store routing keys only
- Leaf nodes store the actual records
- Leaf nodes are linked together for ordered scans
- Split, borrow, and merge logic preserve tree balance
- Internal keys are refreshed after structural changes

```python
tree = BPlusTree(degree=4)

facilities = [
    (1, "Football Ground"),
    (2, "Basketball Court"),
    (3, "Cricket Stadium"),
    (4, "Badminton Hall"),
    (5, "Swimming Pool"),
```

```python
]

for facility_id, name in facilities:
    tree.insert(
        facility_id,
        {
            "Facility_ID": facility_id,
            "Facility_Name": name,
            "Status": "Available",
        },
    )

print("All facilities:")
for key, value in tree.get_all():
    print(f"  {key}: {value['Facility_Name']} ({value['Status']})")

found, record = tree.search(3)
print("\nSearch key 3:", found, record)

tree.update(
    3,
    {
        "Facility_ID": 3,
        "Facility_Name": "Cricket Stadium",
        "Status": "Maintenance",
    },
)
print("Updated key 3:", tree.search(3)[1])

print("\nRange query [2, 4]:")
for key, value in tree.range_query(2, 4):
    print(f"  {key}: {value['Facility_Name']}")

tree.delete(5)
print("\nKeys after deleting 5:", [key for key, _ in tree.get_all()])
```

```
All facilities:
  1: Football Ground (Available)
  2: Basketball Court (Available)
  3: Cricket Stadium (Available)
  4: Badminton Hall (Available)
  5: Swimming Pool (Available)

Search key 3: True {'Facility_ID': 3, 'Facility_Name': 'Cricket
Stadium', 'Status': 'Available'}
Updated key 3: {'Facility_ID': 3, 'Facility_Name': 'Cricket Stadium',
'Status': 'Maintenance'}

Range query [2, 4]:
  2: Basketball Court
```

```
  3: Cricket Stadium
  4: Badminton Hall

Keys after deleting 5: [1, 2, 3, 4]
```

4. How Correctness of Operations Is Ensured

Correctness is enforced at four layers.

4.1 B+ Tree invariants

- keys remain sorted inside each node
- all leaves stay at the same depth
- node occupancy is repaired by split, borrow, and merge logic
- routing keys are refreshed after structural changes

4.2 Table schema validation

`Module_A/table.py` validates every record before insert or update:

- every required column must be present
- every value must match the declared type
- invalid data is rejected before the B+ Tree is modified

4.3 Transaction ordering

`Module_A/transaction.py` follows write-ahead logging:

1. write the log record first
2. flush it with `fsync()`
3. apply the change to the in-memory table
4. record inverse information in the undo log

This ordering is what makes rollback and recovery correct.

4.4 Module B business rules

The Flask layer adds domain-level checks, such as:

- facility status must be one of `Available`, `Maintenance`, or `Closed`
- booking times must be valid
- booking requests must not overlap for the same facility
- references to members and facilities must exist

```
db = DatabaseManager()
db.create_database("demo")
db.create_table(
    "demo",
    "Facility",
```

```
    {"Facility_ID": "int", "Facility_Name": "str", "Status": "str"},
    search_key="Facility_ID",
)

facility_table = db.get_table("demo", "Facility")
facility_table.insert(
    {"Facility_ID": 1, "Facility_Name": "Football Ground", "Status":
"Available"}
)
print("Valid insert succeeded.")

try:
    facility_table.insert({"Facility_ID": 2, "Status": "Available"})
except Exception as exc:
    print("Missing column caught:", exc)

try:
    facility_table.insert(
        {"Facility_ID": "three", "Facility_Name": "Court X", "Status":
"Available"}
    )
except Exception as exc:
    print("Type mismatch caught:", exc)

facility_table.update(
    1,
    {"Facility_ID": 1, "Facility_Name": "Football Ground", "Status":
"Maintenance"},
)
print("Updated record:", facility_table.get(1))
```

```
Database 'demo' created.
Table 'Facility' created in database 'demo'.
Valid insert succeeded.
Missing column caught: [Facility] Missing column 'Facility_Name'
Type mismatch caught: [Facility] 'Facility_ID' must be int, got str
Updated record: {'Facility_ID': 1, 'Facility_Name': 'Football Ground',
'Status': 'Maintenance'}
```

5. How Failures Are Handled

5.1 Failures handled directly during execution

- validation errors raise exceptions before any write reaches the tree
- missing keys or missing tables raise clear Python exceptions
- mid-transaction application failures are handled by `rollback()`

5.2 Crash handling with recovery

`recover()` in `Module_A/transaction.py` reads the WAL and separates transactions into:

- **committed transactions**: redo their operations
- **loser transactions**: undo their operations in reverse order

This mirrors the standard analysis, redo, and undo structure used in ARIES-style recovery.

5.3 Why this works

- committed work survives process failure because the WAL is flushed before data changes
- incomplete work is reversed because each update stores enough before-image information
- rollback and crash recovery both apply inverse operations in reverse order

```
wal_path = temp_wal_path()

db2 = DatabaseManager()
db2.wal.filepath = wal_path
db2.create_database("club")
db2.create_table(
    "club",
    "Member",
    {
        "Member_ID": "str",
        "Name": "str",
        "Age": "int",
        "Gender": "str",
        "Email": "str",
        "Phone_Number": "str",
        "DOB": "str",
        "Image": "str",
    },
    search_key="Member_ID",
)
db2.create_table(
    "club",
    "Facility",
    {
        "Facility_ID": "int",
        "Facility_Name": "str",
        "Facility_Description": "str",
        "Status": "str",
    },
    search_key="Facility_ID",
)

db2.get_table("club", "Facility").insert(
    {
        "Facility_ID": 1,
        "Facility_Name": "Football Ground",
        "Facility_Description": "Grass field",
        "Status": "Available",
```

```
    }
)

txn = db2.begin_transaction()
txn.begin()
txn.insert(
    "club",
    "Member",
    {
        "Member_ID": "M01",
        "Name": "Rahul Sharma",
        "Age": 18,
        "Gender": "M",
        "Email": "rahul@iitgn.ac.in",
        "Phone_Number": "9000000001",
        "DOB": "",
        "Image": "",
    },
)
txn.update(
    "club",
    "Facility",
    1,
    {
        "Facility_ID": 1,
        "Facility_Name": "Football Ground",
        "Facility_Description": "Grass field",
        "Status": "Maintenance",
    },
)
txn.commit()

print("Committed members:", [value["Member_ID"] for _, value in
db2.get_table("club", "Member").get_all()])
print("Facility status after commit:", db2.get_table("club",
"Facility").get(1)["Status"])

txn2 = db2.begin_transaction()
txn2.begin()
try:
    txn2.insert(
        "club",
        "Member",
        {
            "Member_ID": "M02",
            "Name": "Ghost User",
            "Age": 99,
            "Gender": "M",
            "Email": "ghost@iitgn.ac.in",
            "Phone_Number": "0000000000",
```

```
            "DOB": "",
            "Image": "",
        },
    )
    txn2.update(
        "club",
        "Facility",
        1,
        {
            "Facility_ID": 1,
            "Facility_Name": "Football Ground",
            "Facility_Description": "Grass field",
            "Status": "Closed",
        },
    )
    raise RuntimeError("Simulated failure during transaction")
except RuntimeError as exc:
    print("Caught:", exc)
    txn2.rollback()

member_ids = [value["Member_ID"] for _, value in db2.get_table("club",
"Member").get_all()]
facility_status = db2.get_table("club", "Facility").get(1)["Status"]
print("Members after rollback:", member_ids)
print("Facility status after rollback:", facility_status)

cleanup_path(wal_path)
```

```
Database 'club' created.
Table 'Member' created in database 'club'.
Table 'Facility' created in database 'club'.
[TXN T1] BEGIN
  [T1] INSERT -> Member, key=M01
  [T1] UPDATE -> Facility, key=1
[TXN T1] COMMIT
Committed members: ['M01']
Facility status after commit: Maintenance
[TXN T2] BEGIN
  [T2] INSERT -> Member, key=M02
  [T2] UPDATE -> Facility, key=1
Caught: Simulated failure during transaction
[TXN T2] ROLLBACK — undoing 2 operation(s)...
  Undid UPDATE: restored key=1 in Facility
  Undid INSERT: deleted key=M02 from Member
[TXN T2] ROLLBACK complete ✓
Members after rollback: ['M01']
Facility status after rollback: Maintenance
```

```
wal_path = temp_wal_path()
```

```
db3 = DatabaseManager()
db3.wal.filepath = wal_path
db3.create_database("waltest")
db3.create_table(
    "waltest",
    "Facility",
    {
        "Facility_ID": "int",
        "Facility_Name": "str",
        "Facility_Description": "str",
        "Status": "str",
    },
    search_key="Facility_ID",
)

table = db3.get_table("waltest", "Facility")
table.insert(
    {
        "Facility_ID": 10,
        "Facility_Name": "Tennis Court",
        "Facility_Description": "Clay",
        "Status": "Available",
    }
)

txn = db3.begin_transaction()
txn.begin()
txn.update(
    "waltest",
    "Facility",
    10,
    {
        "Facility_ID": 10,
        "Facility_Name": "Tennis Court",
        "Facility_Description": "Clay",
        "Status": "Maintenance",
    },
)
txn.commit()

print(f"{'LSN':<5} {'TXN':<6} {'OP':<10} {'TABLE':<12} {'KEY':<6}
DETAIL")
print("-" * 80)
for index, entry in enumerate(db3.wal.read_all(), start=1):
    detail = ""
    if entry.get("op") == "UPDATE":
        before = entry.get("old_value", {}).get("Status")
        after = entry.get("new_value", {}).get("Status")
        detail = f"before={before!r}, after={after!r}"
    print(
```

```
        f"{index:<5} {entry.get('txn_id', ''):<6} {entry.get('op',
''):<10} "
        f"{entry.get('table', ''):<12} {str(entry.get('key', '')):<6}
{detail}"
    )

cleanup_path(wal_path)
```

```
Database 'waltest' created.
Table 'Facility' created in database 'waltest'.
[TXN T3] BEGIN
  [T3] UPDATE -> Facility, key=10
[TXN T3] COMMIT
LSN   TXN    OP         TABLE        KEY    DETAIL
----------------------------------------------------------------------
----------
1     T3     BEGIN
2     T3     UPDATE     Facility     10     before='Available',
after='Maintenance'
3     T3     COMMIT
```

```
from bplustree import BPlusTree

wal_path = temp_wal_path()

db_recover = DatabaseManager()
db_recover.wal.filepath = wal_path
db_recover.create_database("sports")
db_recover.create_table(
    "sports",
    "Booking",
    {
        "Booking_ID": "int",
        "Facility_ID": "int",
        "Member_ID": "str",
        "Time_In": "str",
        "Time_Out": "str",
    },
    search_key="Booking_ID",
)

booking_table = db_recover.get_table("sports", "Booking")

txn = db_recover.begin_transaction()
txn.begin()
txn.insert(
    "sports",
    "Booking",
    {
        "Booking_ID": 101,
```

```
        "Facility_ID": 1,
        "Member_ID": "M01",
        "Time_In": "2026-04-10 09:00",
        "Time_Out": "2026-04-10 10:00",
    },
)
txn.commit()

print("Before simulated restart:", booking_table.get(101))
booking_table.tree = BPlusTree(4)
print("After wiping in-memory tree:", booking_table.get(101))

summary = db_recover.recover()
print("Recovery summary:", summary)
print("After recovery:", booking_table.get(101))

cleanup_path(wal_path)
```

```
Database 'sports' created.
Table 'Booking' created in database 'sports'.
[TXN T4] BEGIN
  [T4] INSERT -> Booking, key=101
[TXN T4] COMMIT
Before simulated restart: {'Booking_ID': 101, 'Facility_ID': 1,
'Member_ID': 'M01', 'Time_In': '2026-04-10 09:00', 'Time_Out': '2026-
04-10 10:00'}
After wiping in-memory tree: None
[RECOVERY] T4 — COMMITTED, redoing 1 op(s)...
[RECOVERY] T4 — Redo complete ✓
Recovery summary: {'status': 'recovered', 'records': 3, 'committed':
['T4'], 'undone': []}
After recovery: {'Booking_ID': 101, 'Facility_ID': 1, 'Member_ID':
'M01', 'Time_In': '2026-04-10 09:00', 'Time_Out': '2026-04-10 10:00'}
```

```
wal_path = temp_wal_path()

db_loser = DatabaseManager()
db_loser.wal.filepath = wal_path
db_loser.create_database("sports2")
db_loser.create_table(
    "sports2",
    "Facility",
    {
        "Facility_ID": "int",
        "Facility_Name": "str",
        "Facility_Description": "str",
        "Status": "str",
    },
    search_key="Facility_ID",
)
```

```
facility_table = db_loser.get_table("sports2", "Facility")
facility_table.insert(
    {
        "Facility_ID": 1,
        "Facility_Name": "Football Ground",
        "Facility_Description": "Grass field",
        "Status": "Available",
    }
)

db_loser.wal.log_begin("T_LOSER")
db_loser.wal.log_update(
    "T_LOSER",
    "sports2",
    "Facility",
    1,
    {
        "Facility_ID": 1,
        "Facility_Name": "Football Ground",
        "Facility_Description": "Grass field",
        "Status": "Available",
    },
    {
        "Facility_ID": 1,
        "Facility_Name": "Football Ground",
        "Facility_Description": "Grass field",
        "Status": "Closed",
    },
)
facility_table.update(
    1,
    {
        "Facility_ID": 1,
        "Facility_Name": "Football Ground",
        "Facility_Description": "Grass field",
        "Status": "Closed",
    },
)

print("Before recovery:", facility_table.get(1))
summary = db_loser.recover()
print("Recovery summary:", summary)
print("After recovery:", facility_table.get(1))

cleanup_path(wal_path)
```

```
Database 'sports2' created.
Table 'Facility' created in database 'sports2'.
Before recovery: {'Facility_ID': 1, 'Facility_Name': 'Football
```

```
Ground', 'Facility_Description': 'Grass field', 'Status': 'Closed'}
[RECOVERY] T_LOSER — NO COMMIT found, undoing 1 op(s)...
  Recovery: restored key=1 in Facility
[RECOVERY] T_LOSER — Undo complete ✓
Recovery summary: {'status': 'recovered', 'records': 2, 'committed':
[], 'undone': ['T_LOSER']}
After recovery: {'Facility_ID': 1, 'Facility_Name': 'Football Ground',
'Facility_Description': 'Grass field', 'Status': 'Available'}
```

6. How Multi-User Conflicts Are Handled

6.1 Locking strategy

Conflict handling is based on shared module-level `threading.RLock` objects keyed by `db_name.table_name`.

- a transaction acquires the table lock before changing that table
- locks are held until `commit()` or `rollback()`
- this is a strict two-phase locking pattern at table granularity

6.2 Deadlock avoidance in Module B

`Module_B/web/database.py` adds `_acquire(*table_names)` and `_release(*table_names)` helpers.
When multiple tables must be locked together, table names are sorted before acquisition.
This consistent ordering reduces circular wait risk for the fixed set of tables used by the web API.

6.3 What conflicts are handled well in this project

- competing writes on the same table
- lost updates in the demonstrated booking workflow
- inconsistent multi-table writes when one request fails partway through

6.4 Important limitation

The implementation uses **table-level** locks, not row-level locks.
That means concurrency is safe, but write throughput is reduced when many users target the same table.

Also, the engine does **not** implement a full MVCC or full read-locking model.
So the strongest accurate claim is that it prevents the write conflicts demonstrated in the assignment workload, not that it is a production-grade serialisable system.

```
wal_path = temp_wal_path()

db4 = DatabaseManager()
db4.wal.filepath = wal_path
db4.create_database("iso")
db4.create_table(
```

```
    "iso",
    "Facility",
    {
        "Facility_ID": "int",
        "Facility_Name": "str",
        "Facility_Description": "str",
        "Status": "str",
    },
    search_key="Facility_ID",
)

facility_table = db4.get_table("iso", "Facility")
facility_table.insert(
    {
        "Facility_ID": 1,
        "Facility_Name": "Football Ground",
        "Facility_Description": "Grass field",
        "Status": "Available",
    }
)

events = []


def writer_txn():
    txn = db4.begin_transaction()
    txn.begin()
    txn.update(
        "iso",
        "Facility",
        1,
        {
            "Facility_ID": 1,
            "Facility_Name": "Football Ground",
            "Facility_Description": "Grass field",
            "Status": "Maintenance",
        },
    )
    events.append("Writer updated status to Maintenance")
    time.sleep(0.3)
    txn.rollback()
    events.append("Writer rolled back")


def reader_after_writer():
    time.sleep(0.35)
    events.append(f"Reader sees status = {facility_table.get(1)
['Status']}")
```

```
t1 = threading.Thread(target=writer_txn)
t2 = threading.Thread(target=reader_after_writer)
t1.start()
t2.start()
t1.join()
t2.join()

print("Isolation demo:")
for item in events:
    print(" ", item)

cleanup_path(wal_path)
```

```
Database 'iso' created.
Table 'Facility' created in database 'iso'.
[TXN T5] BEGIN
  [T5] UPDATE -> Facility, key=1
[TXN T5] ROLLBACK — undoing 1 operation(s)...
  Undid UPDATE: restored key=1 in Facility
[TXN T5] ROLLBACK complete ✓
Isolation demo:
  Writer updated status to Maintenance
  Writer rolled back
  Reader sees status = Available
```

```
wal_path = temp_wal_path()

db5 = DatabaseManager()
db5.wal.filepath = wal_path
db5.create_database("race")
db5.create_table(
    "race",
    "Booking",
    {
        "Booking_ID": "int",
        "Facility_ID": "int",
        "Member_ID": "str",
        "Time_In": "str",
        "Time_Out": "str",
    },
    search_key="Booking_ID",
)
db5.create_table(
    "race",
    "Facility",
    {
        "Facility_ID": "int",
        "Facility_Name": "str",
        "Facility_Description": "str",
        "Status": "str",
```

```
    },
    search_key="Facility_ID",
)

facility_table = db5.get_table("race", "Facility")
booking_table = db5.get_table("race", "Booking")

facility_table.insert(
    {
        "Facility_ID": 1,
        "Facility_Name": "Football Ground",
        "Facility_Description": "Grass field",
        "Status": "Available",
    }
)

results = []
results_lock = threading.Lock()


def try_book(worker_id):
    txn = db5.begin_transaction()
    txn.begin()
    try:
        locks = [_get_lock("race", name) for name in
sorted({"Booking", "Facility"})]
        for lock in locks:
            lock.acquire()
        try:
            current = facility_table.get(1)
            if current["Status"] != "Available":
                raise RuntimeError("Slot already taken")

            txn.insert(
                "race",
                "Booking",
                {
                    "Booking_ID": worker_id,
                    "Facility_ID": 1,
                    "Member_ID": f"M{worker_id:02d}",
                    "Time_In": "2026-04-10 09:00",
                    "Time_Out": "2026-04-10 10:00",
                },
            )
            txn.update(
                "race",
                "Facility",
                1,
                {
                    "Facility_ID": 1,
```

```
                    "Facility_Name": "Football Ground",
                    "Facility_Description": "Grass field",
                    "Status": "Maintenance",
                },
            )
            txn.commit()
            outcome = ("commit", worker_id)
        finally:
            for lock in reversed(locks):
                lock.release()
    except Exception as exc:
        if txn.active:
            txn.rollback()
        outcome = ("reject", worker_id, str(exc))

    with results_lock:
        results.append(outcome)


threads = [threading.Thread(target=try_book, args=(idx,)) for idx in
range(1, 7)]
for thread in threads:
    thread.start()
for thread in threads:
    thread.join()

commits = [item for item in results if item[0] == "commit"]
rejects = [item for item in results if item[0] == "reject"]

print("Committed bookings:", commits)
print("Rejected attempts:", rejects)
print("Final bookings in tree:", len(booking_table.get_all()))
print("Final facility status:", facility_table.get(1)["Status"])

cleanup_path(wal_path)
```

```
Database 'race' created.
Table 'Booking' created in database 'race'.
Table 'Facility' created in database 'race'.
[TXN T6] BEGIN
[TXN T7] BEGIN
[TXN T8] BEGIN
[TXN T9] BEGIN
[TXN T10] BEGIN
[TXN T11] BEGIN
  [T6] INSERT -> Booking, key=1
  [T6] UPDATE -> Facility, key=1
[TXN T6] COMMIT
[TXN T7] ROLLBACK — undoing 0 operation(s)...
[TXN T8] ROLLBACK — undoing 0 operation(s)...
```

```
[TXN T9] ROLLBACK — undoing 0 operation(s)...
[TXN T10] ROLLBACK — undoing 0 operation(s)...
[TXN T11] ROLLBACK — undoing 0 operation(s)...
[TXN T7] ROLLBACK complete ✓
[TXN T8] ROLLBACK complete ✓
[TXN T9] ROLLBACK complete ✓
[TXN T10] ROLLBACK complete ✓
[TXN T11] ROLLBACK complete ✓
Committed bookings: [('commit', 1)]
Rejected attempts: [('reject', 2, 'Slot already taken'), ('reject', 3, 'Slot already taken'), ('reject', 4, 'Slot already taken'), ('reject', 5, 'Slot already taken'), ('reject', 6, 'Slot already taken')]
Final bookings in tree: 1
Final facility status: Maintenance
```

7. Experiments Performed

The project contains both in-notebook demonstrations and web-layer experiments in Module B.

7.1 Experiments completed

1. **B+ Tree functional demo**
   Insert, search, update, delete, and range query operations.

2. **Schema validation test**
   Verified that missing columns and wrong types are rejected.

3. **Atomicity test**
   Forced an exception in the middle of a transaction and confirmed rollback restored the previous state.

4. **Durability and recovery test**
   Committed a transaction, simulated a restart, and verified `recover()` restored the committed data.

5. **Loser transaction recovery test**
   Simulated an uncommitted update in the WAL and verified recovery undid it.

6. **Isolation test**
   Demonstrated that conflicting activity on the same table does not leave a dirty final state.

7. **Concurrent booking race test**
   Multiple worker threads attempted to book the same facility; only one booking committed.

8. **Module B load testing**
   The repository also includes a real Locust workload in `Module_B/stress/locustfile.py`.

```
experiments = [
    {
        "experiment": "Atomicity crash simulation",
        "result": "pass",
        "observation": "Rollback removed every partial write from the
failed transaction.",
    },
    {
        "experiment": "Consistency validation",
        "result": "pass",
        "observation": "Bad records were rejected before they reached
the tree or WAL.",
    },
    {
        "experiment": "Durability recovery",
        "result": "pass",
        "observation": "Committed changes were reconstructed from the
WAL after the in-memory tree was wiped.",
    },
    {
        "experiment": "Concurrent booking race",
        "result": "pass",
        "observation": "Only one booking committed for the single
available slot.",
    },
]

print("Experiment summary:")
for item in experiments:
    print(f"- {item['experiment']}: {item['result'].upper()} |
{item['observation']}")
```

```
Experiment summary:
- Atomicity crash simulation: PASS | Rollback removed every partial
write from the failed transaction.
- Consistency validation: PASS | Bad records were rejected before they
reached the tree or WAL.
- Durability recovery: PASS | Committed changes were reconstructed
from the WAL after the in-memory tree was wiped.
- Concurrent booking race: PASS | Only one booking committed for the
single available slot.
```

8. Observations

- The design is easiest to reason about because the WAL, undo log, and table locks all follow a clear order.
- Correctness is strong for the assignment-scale workload because every update path is validated before or during modification.
- Recovery is practical because the WAL stores enough information to replay committed work and reverse incomplete work.

- Coarse table locks make the system safer to implement, but they also reduce concurrency under write-heavy workloads.
- The demonstrations show that the engine behaves predictably even when failures are injected deliberately.

9. Limitations

The implementation works for the assignment goals, but it still has important limits.

1. **Table-level locking only**
   Concurrent writers to the same table are serialised, so throughput drops under contention.

2. **No checkpointing or WAL truncation**
   Recovery time will grow as the WAL becomes longer.

3. **In-memory tree only**
   The B+ Tree itself is not stored on disk; the WAL is the only persistence mechanism.

4. **Single WAL append lock**
   Every logged write waits on the same WAL file lock and `fsync()`.

5. **No deadlock detection**
   Module B avoids deadlocks through consistent lock ordering, but the engine does not detect arbitrary cycles.

6. **Locust file still uses old relation names**
   `Module_B/stress/locustfile.py` currently checks for `["Orders", "Products", "Users"]`, while the actual tables are `Member`, `Facility`, `Booking`, and `Complaint`. This should be corrected before presenting load-test results from that script.

10. Conclusion

This assignment delivers a layered mini-database system rather than a single isolated data structure.

- **Module A** provides the B+ Tree, schema-checked tables, WAL, transactions, rollback, and recovery.
- **Module B** exposes those guarantees through a Flask API and concurrency demonstrations.

Overall, the report evidence supports the following conclusion:

- operations are kept correct through validation, invariants, and WAL-before-write ordering
- failures are handled through rollback and crash recovery
- multi-user conflicts are handled through table-level locking and ordered lock acquisition

- the experiments demonstrate that the design works, while the limitations make clear where the system would need improvement for production use